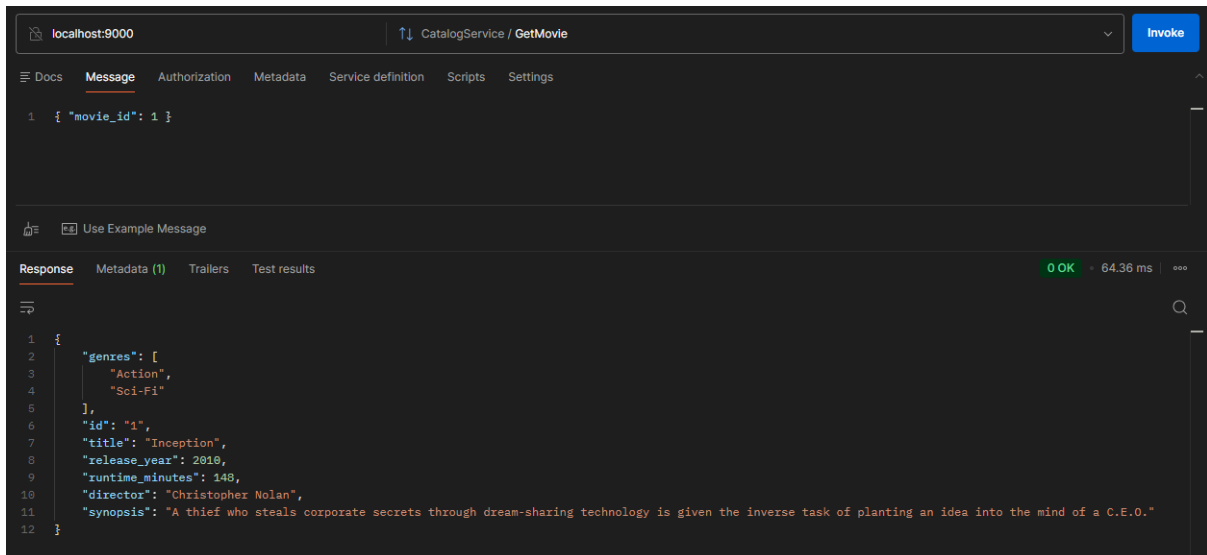
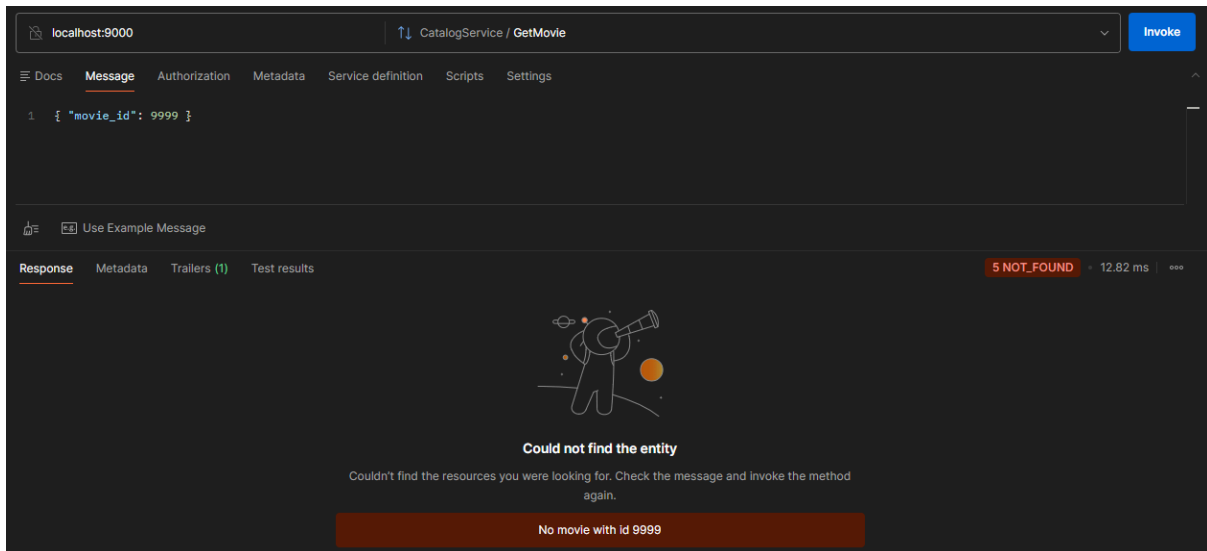
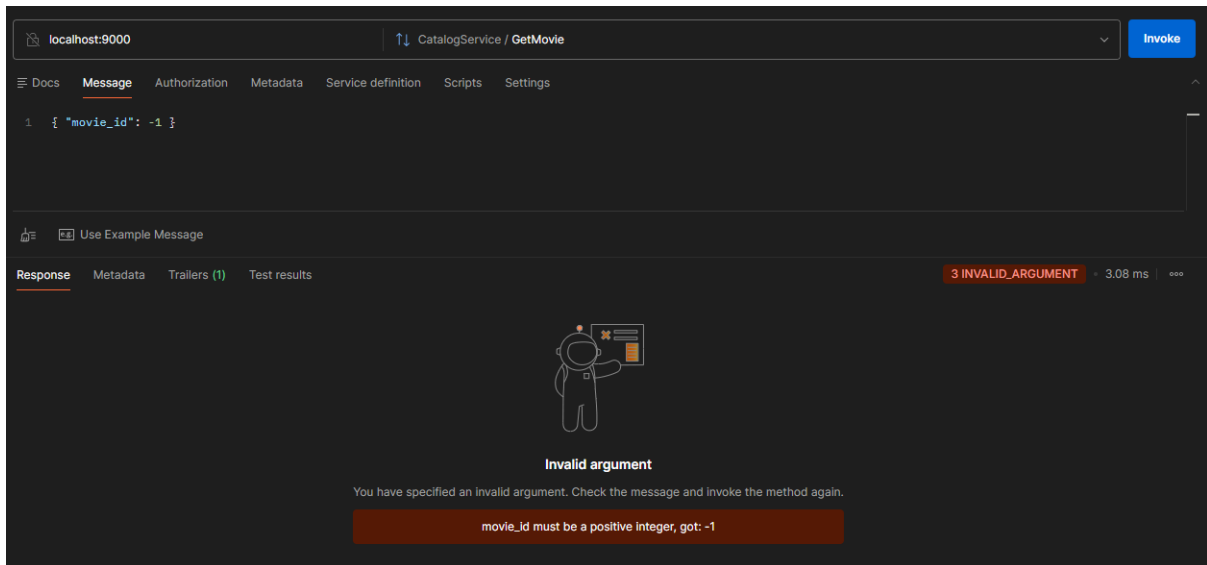




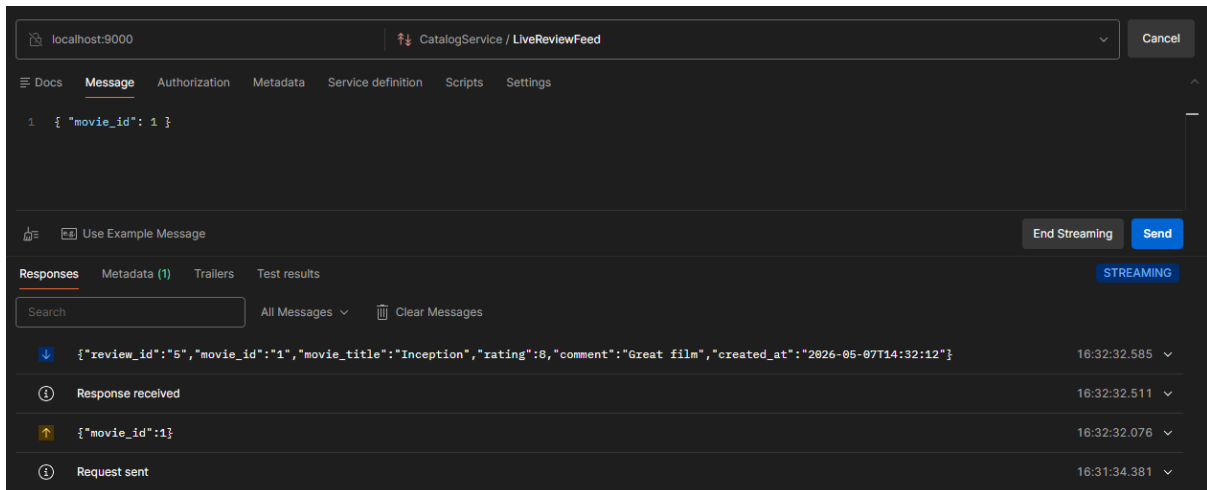
Test 1 — GetMovie (positive) Sent a valid `movie_id: 1`. The server returned status **OK** with the full movie details for *Inception*, including its two genres (Action, Sci-Fi). Demonstrates that the unary RPC correctly fetches data from the SQLite database and serializes it through Protobuf.



Test 2 — GetMovie (not found) Sent a non-existent `movie_id: 9999`. The server returned status `NOT_FOUND` with the message *"No movie with id 9999"*. Demonstrates correct error handling using gRPC's standard status codes when the requested resource does not exist.



Test 3 — GetMovie (invalid argument) Sent a negative `movie_id: -1`. The server returned status `INVALID_ARGUMENT` with the message *"movie_id must be a positive integer, got: -1"*. Demonstrates that input validation runs before the database is queried, rejecting malformed requests early.

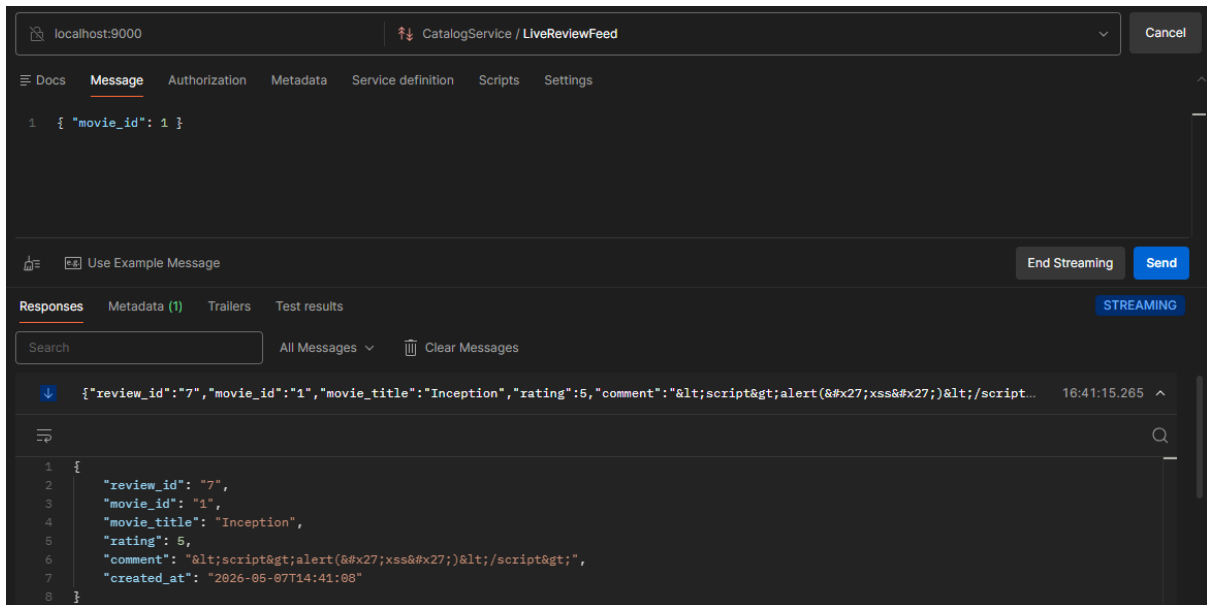


Test 4 — LiveReviewFeed (positive) Subscribed to `movie_id: 1` over the bidirectional stream. A new review was inserted into the database via `add_test_review.py`, and the server pushed the corresponding `ReviewUpdate` message to the client within 2 seconds. Demonstrates that the streaming RPC stays open and delivers live updates as data changes.

The screenshot shows a REST client interface with a dark theme. At the top, the address bar shows 'localhost:9000' and the selected resource is 'CatalogService / LiveReviewFeed'. Below the address bar, there are tabs for 'Docs', 'Message', 'Authorization', 'Metadata', 'Service definition', 'Scripts', and 'Settings'. The 'Message' tab is active, displaying a JSON body: `{ "movie_id": 0 }`. Below the message editor, there are buttons for 'Use Example Message', 'End Streaming', and 'Send'. The 'Responses' tab is also visible, showing a list of messages. The first message is `{"movie_id":0}` with a timestamp of 16:33:45.284. Below the responses, there is a server log showing the following messages:

```
2026-05-07 16:32:32,077 [INFO] Subscribed to movie_id=1
2026-05-07 16:33:45,284 [WARNING] Invalid subscribe request: movie_id must be a positive integer, got: 0
```

Test 5 — LiveReviewFeed (invalid argument) Sent a subscribe request with `movie_id: 0`. No review was streamed back, and the server log shows *"WARNING: Invalid subscribe request: movie_id must be a positive integer, got: 0"*. Demonstrates that invalid stream messages are silently ignored without terminating the connection, allowing the client to recover by sending a valid subscribe afterwards.



Test 6 — XSS sanitization Inserted a review containing `<script>alert('xss')</script>` into the database. When streamed back, every HTML special character was escaped (`<` → `<`; `'` → `'`; `>` → `>`), making the script tag inert. Any downstream consumer that renders the comment as HTML will display it as literal text instead of executing it. Demonstrates server-side XSS prevention.